Universidad Autónoma de Yucatán

Facultad de Matemáticas

Licenciatura en Ingeniería de Software virtual

Arquitectura y Organización de Computadoras

Profesor: **Miguel Angel Canul Chin**

Grupo: B

Anteproyecto

Equipo #15 (individual)

Solís Canul Elda Alejandra

Domingo 04 Octubre 2026

Mérida, Yucatán, México

Enlace al repositorio del simulador del sistema automatizado de monitoreo y control de temperatura: https://github.com/SolisDg/AnteproyectoAOC_Equipo-15_Solis.git

Enlace al simulador didáctico sobre el ciclo de instrucción:

Vista: https://canva.link/nvpmgxg4zcm58e1

Enlace a la actividad 2 del canva (en caso de que falle el enlace ahí proporcionado: https://sites.google.com/view/simulador-didctico/

Nombre del anteproyecto: Sistema automatizado de monitoreo y control de temperatura mediante un microcontrolador

Introducción:

Los microcontroladores nos permiten desarrollar sistemas capaces de recibir informaición de su entorno, procesarla mediante instrucciones programadas (como veremos en este anteproyecto) y generar una respuesta de manera automática, lo que hace posible su aplicación en diferentes contextos y situaciones donde se requiere monitorear en tiempo real determinadas condiciones (como la tempratura en nuestro caso) y actuar de alguna forma determinada cuando se alcanzan valores previamente establecidos en su programación

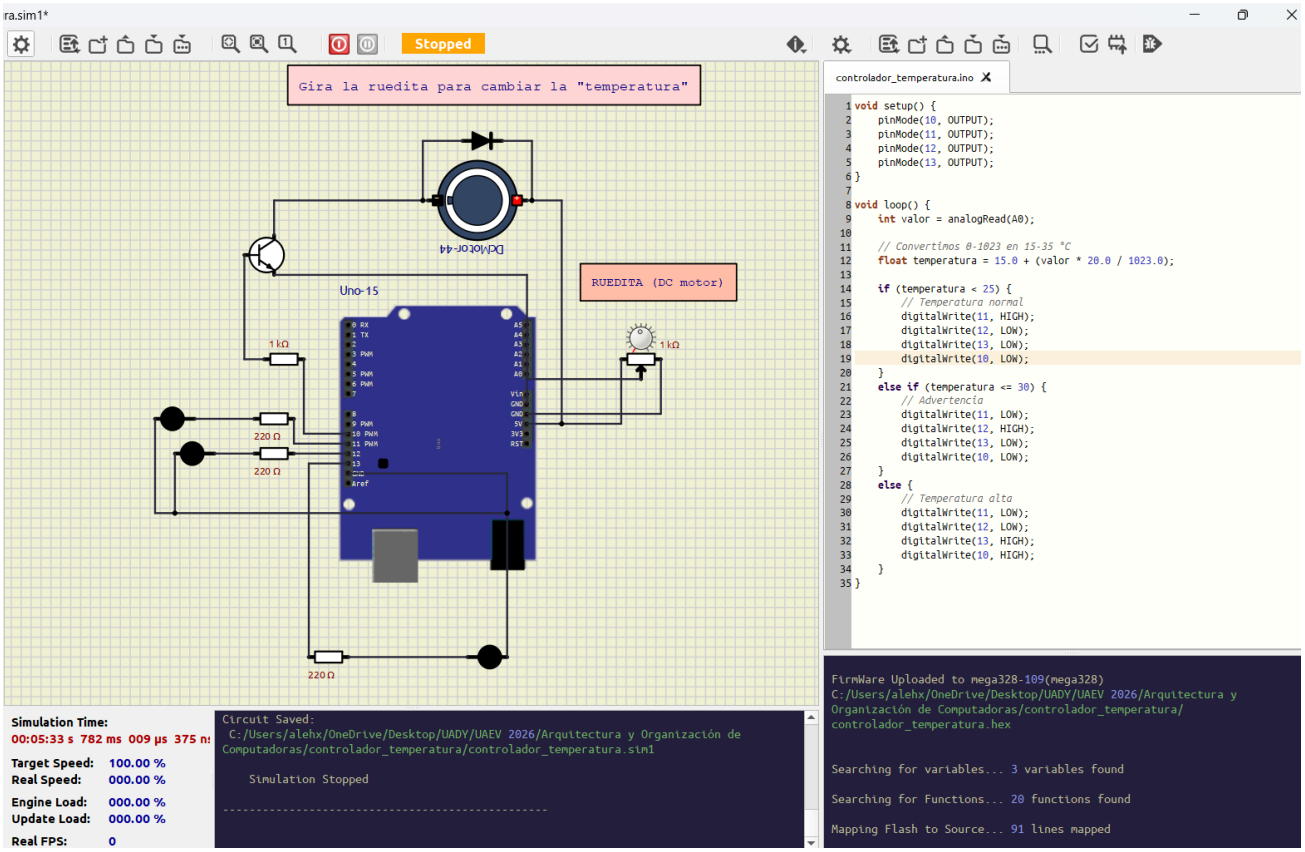
Una diferencia importante que queremos mencionar es la diferencia entre el microprocesador y el microcontrolador, el microprocesador es principalmente una unidad de procesamiento con capacidad limitada para procesar el entorno, mientras que el microcontrolador tiene integrado tanto el procesamiento como las herramientas necesarias para interactuar con el exterior, justo lo que necesitábamos para desarrollar este proyecto.

Otra nota importante es que el Arduino UNO es la placa que utiliza el microcontrolador ATmega328P.

El presente proyecto propone el desarrollo de un sistema automatizado de monitoreo y control de temperatura mediante un microcontrolador Arduino UNO R3 basado en el ATmega328P, y para demostrar su funcionamiento, el equipo desarrolló un prototipo en SimulIDE con ayuda de Arduino IDE, en el cual un potenciómetro representará la entrada de un sensor de temperatura o termómetro y permitirá generar diferentes valores que serán procesados por el microcontrolador.

Nuestro prototipo busca demostrar la interacción entre entrada, procesamiento y respuesta de un sistema basado en la arquitectura de un microcontrolador, así como proporcionar un contexto de las posibilidades de adaptar este principio a mayor escala mediante sensores y otros sistemas reales.

Nota del alumno*: el proyecto originalmente estaba diseñado para complementar el Juego del Ciclo de Instrucción en colaboración con mi original equipo #2, pero no conseguí que se involucraran en el proyecto, por lo que decidí hacer esta parte sola, tanto la planeación como el desarrollo del prototipo. Trabajé con ellas en el diseño del prototipo del Juego, pero consideré necesario un complemento que aplicara de forma más práctica los conocimientos que adquirimos en esta materia, ambos cumplen propósitos complementarios.*



¿Qué hará?

El sistema simulará una alarma que enciende un ventilador con el cambio de temperatura de la habitación. El potenciómetro hará el papel del “termómetro”, analogRead(A0) puede devolver valores entre 0 y 1023, por lo que usaremos una fórmula que simule las “temperaturas” que detectará nuestro microprocesador, la fórmula está en el código expresada como 15+[(X(20))/1023], siendo 15°C la temperatura más alta que nos arrojará el motor y 35 °C la más alta, la X representa los diferentes valores que nos dará el motor y el 20 es la diferencia entre 15 y 35, las posibles temperaturas que puede arrojarnos.

¿Qué problema busca resolver?

El panorama en el que podemos usar la programación de este microcontrolador y mejorarlo con un mejor hardware y software, es por ejemplo en la agricultura, con las temperaturas actuales los incendios se volverán cada vez más frecuentes, con ayuda de estos sensores programados para activar el termómetro cuando aumenta la temperatura, los agricultores no tienen que preocuparse de sufrir un incendio tan de repente, el ventilador puede ser sustituido por ejemplo por sistemas de riego, de forma que en lugar de que la señal viaje hasta activar un ventilador, active los aspersores de los cultivos.

De igual forma, esto puede servir en las ciudades que cuentan con sistemas detectores de humo para liberar agua, en lugar de depender de una fuente de humo, un calor extremo

sería suficiente para activar los aspersores del jardín, como se vio en 2017 el caso de un hombre en Estados Unidos que salvó su casa de un incendio con sólo encender los aspersores antes de irse. Nuestro prototipo busca ser el inicio de un dispositivo funcional que active los aspersores ante un aumento de la temperatura que rebase los límites normales de una determinada región, o que, con otro tipo de sensores en lugar del potenciómetro, pueda medir la humedad en un huerto y activar los sistemas de riego cuando la humedad descienda de un rango previamente programado, lo contrario a nuestro sistema que se activa ante un aumento, pero siguiendo el mismo principio de activar un segundo mecanismo a raíz de los datos entrantes a un primero.

Objetivo general:

Demostrar que con pocos materiales y un poco de programación, es posible crear una solución a un problema de la vida real, un sistema automatizado de monitoreo y control de temperatura mediante Arduino UNO.

Limitaciones del proyecto:

El uso de un potenciómetro (Motor DC) a modo de termómetro o sensor, la idea es que esto a nivel más alto, pueda conectarse a sensores de temperatura y adaptar los rangos a la temperatura de los lugares donde sea utilizado, no tenemos el mismo rango de temperaturas aquí en Yucatán que en Baja California o Ciudad de México por ejemplo.

Materiales utilizados:

Software: SimuladorIDE 1.1.0-SR2 y ArduinoIDE 2.3.10

Lenguaje de programación: C++

Microcontrolador: Arduino UNO r3 / ATmega328P

Cantidad	Componente	Valor / especificación	Función
1	**Arduino UNO R3**	ATmega328P	Microcontrolador principal
1	**Potenciómetro**	1 kΩ	Simula la temperatura mediante una entrada analógica
3	**LED**	Verde, amarillo y rojo	Indican el estado de temperatura
3	**Resistencia**	220 Ω	Limitan la corriente de cada LED

1	**Transistor BJT NPN**	Ganancia ≈ 100	Actúa como interruptor para el ventilador
1	**Resistencia**	1 kΩ	Limita la corriente hacia la base del transistor
1	**Motor DC**	Ventilador simulado	Actuador que se enciende cuando la temperatura supera 30 °C
1	**Diodo**	Diodo de protección	Protege frente al voltaje generado por el motor
—	**Cables/conexiones**	—	Interconexión del circuito
—	**5 V y GND**	Arduino UNO	Alimentación y referencia del circuito

Se utilizó un potenciómetro de 1kΩ como entrada analógica (ajustable con solo mover la ruedita), para simular el sensor de temperatura del sistema, de modo que fuera posible observar los cambios ocurridos en tiempo real al modificar el voltaje que recibía el pin A0 del Arduino UNO.

Tendremos:

Parte 1: detección de temperatura (potenciómetro en el prototipo)

Parte 2: activación del ventilador bajo ciertas condiciones (motor DC en el prototipo)

Condiciones:

Temperatura | Interpretación de nuestro sistema

Menor a 25°C -> Condición normal

Entre 25 y 30°C -> Advertencia

Mayor a 30°C -> Activar ventilador y alarma

Ejemplo en el código:

```
if (temperatura < 25) {
   // Normal
}
else if (temperatura <= 30) {
   // Advertencia
}
```

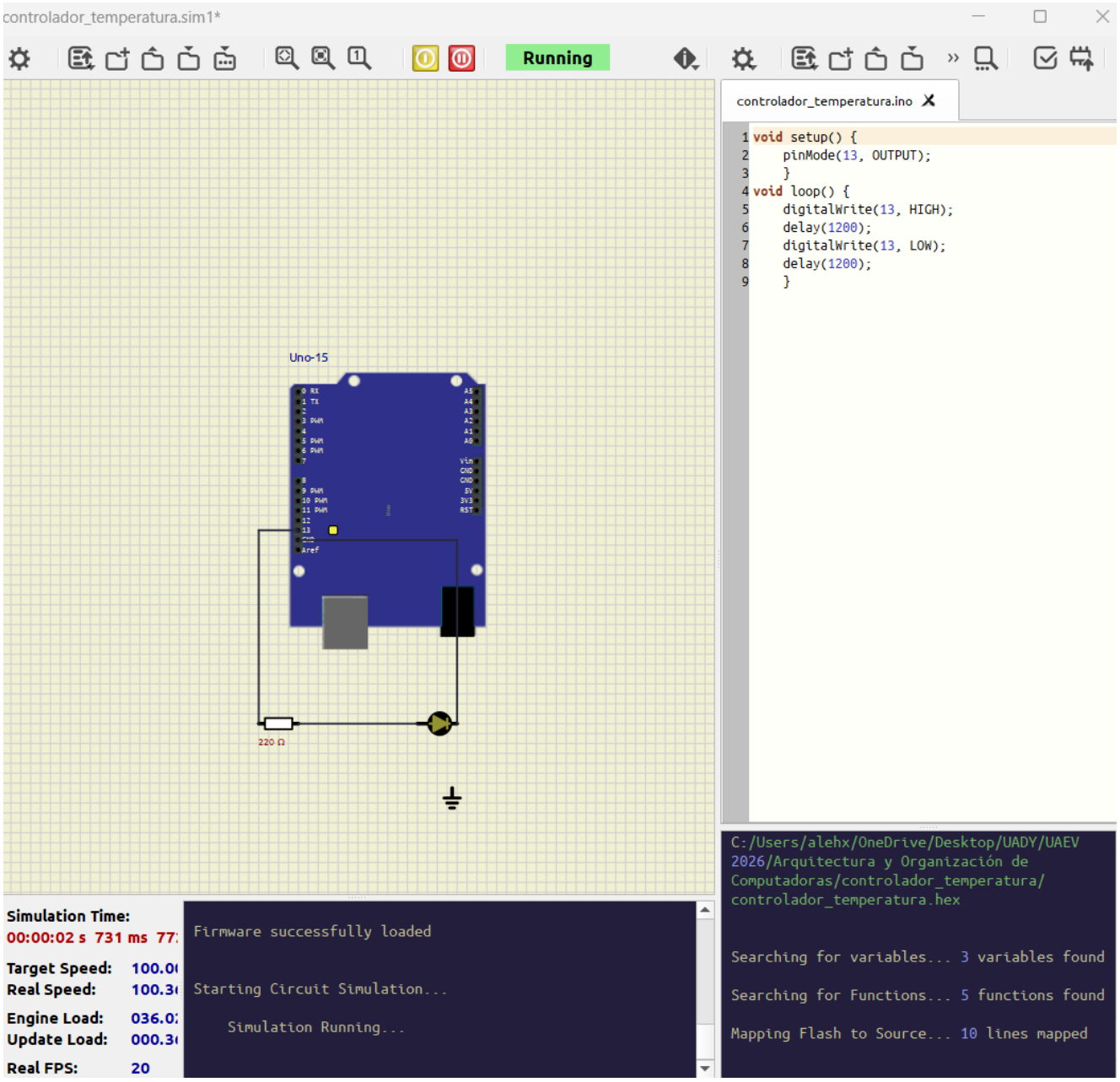
```
else {
    // Alarma – Encendido del ventilador (motor DC)
}
```

Usaremos Arduino UNO basado en el ATmega328P por su facilidad de documentar para el anteproyecto

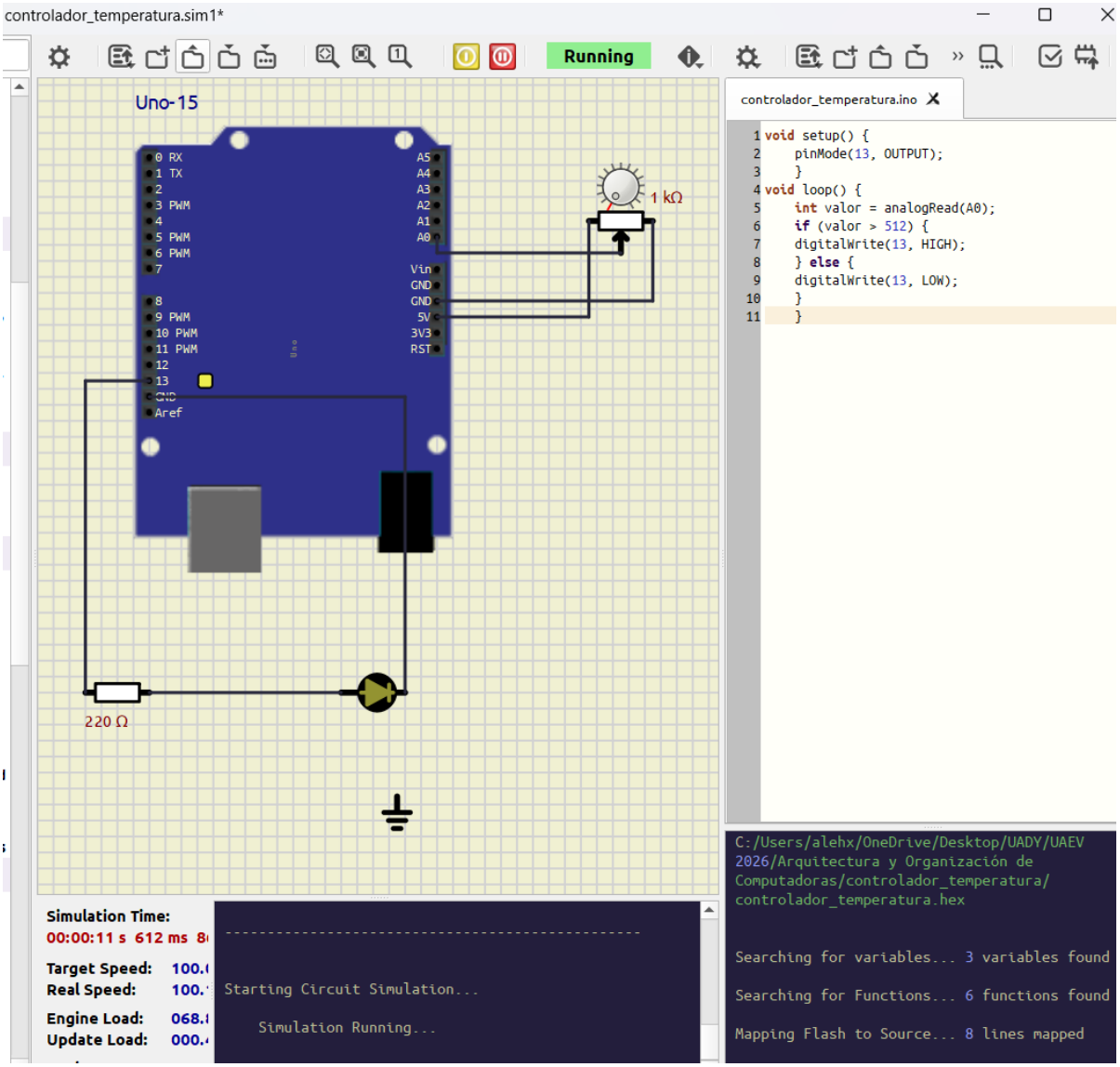
Fases del proyecto:

Fase 1: Arduino + LED
Fase 2: Arduino + sensor
Fase 3: Arduino +sensor + LED
Fase 4: Arduino + sensor + ventilador
Fase 5: pruebas y documentación

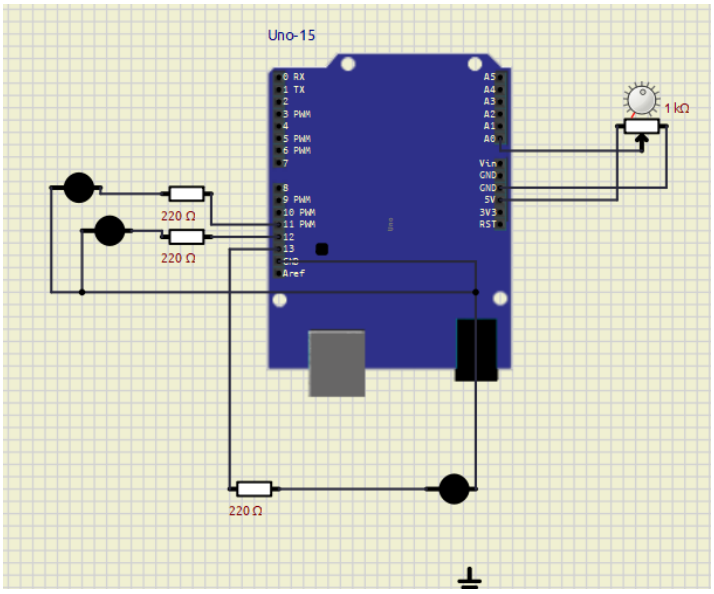
Prueba 1: encender un LED que parpadee a intervalos de 1200 milisegundos para comprobar que funciona:



Prueba 2: comprobamos que el LED sigue funcionando luego de conectar el potenciómetro

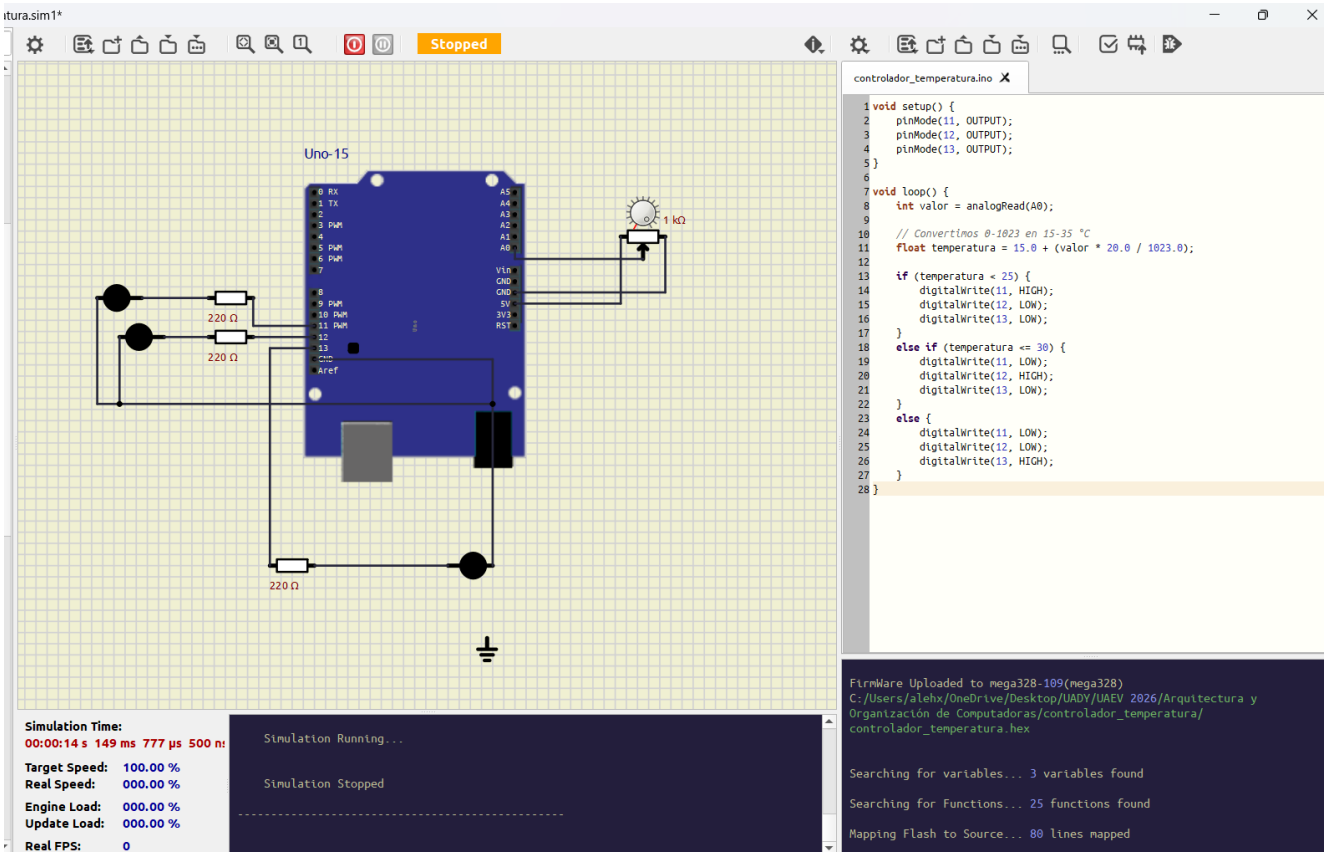


Luego conectamos los LEDs restantes y nos aseguramos que todos tengan una resistencia de 220Ohm para evitar quemar los LED:



Luego le indicamos al microcontrolador que los pines 11, 12, 13 son salidas:

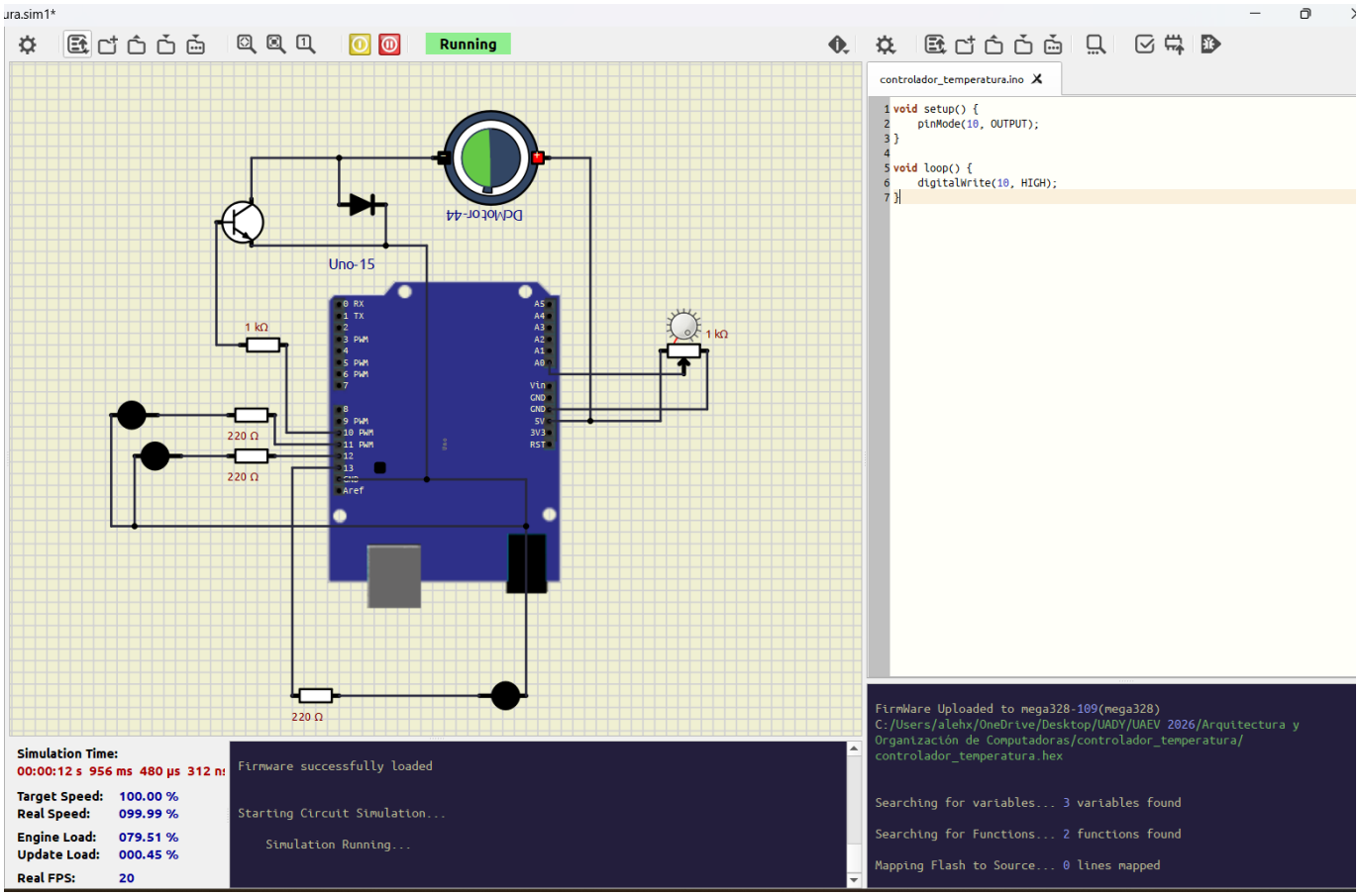
```
void setup() {
pinMode(11, OUTPUT);
pinMode(12, OUTPUT);
pinMode(13, OUTPUT);
}
void loop() {
int valor = analogRead(A0);
//Convertimos 0-1023 del potenciómetro en 15-35 °C para simular la tempratura
float temperatura = 15.0 + (valor * 20.0 / 1023.0);
if (temperatura < 25) {
digitalWrite(11, HIGH);
digitalWrite(12, LOW);
digitalWrite(13, LOW);
}
else if (temperatura <= 30) {
digitalWrite(11, LOW);
digitalWrite(12, HIGH);
digitalWrite(13, LOW);
}
else {
digitalWrite(11, LOW);
digitalWrite(12, LOW);
digitalWrite(13, HIGH);
}
}
```


tura.sim1*
Stopped
Uno-15
220 Ω
220 Ω
220 Ω
1 kΩ
controlador_temperatura.ino
void setup() {
pinMode(11, OUTPUT);
pinMode(12, OUTPUT);
pinMode(13, OUTPUT);
}
void loop() {
int valor = analogRead(A0);
// Convertimos 0-1023 en 15-35 °C
float temperatura = 15.0 + (valor * 20.0 / 1023.0);
if (temperatura < 25) {
digitalWrite(11, HIGH);
digitalWrite(12, LOW);
digitalWrite(13, LOW);
}
else if (temperatura <= 30) {
digitalWrite(11, LOW);
digitalWrite(12, HIGH);
digitalWrite(13, LOW);
}
else {
digitalWrite(11, LOW);
digitalWrite(12, LOW);
digitalWrite(13, HIGH);
}
}
Simulation Time:
00:00:14 s 149 ms 777 µs 500 n
Target Speed: 100.00 %
Real Speed: 000.00 %
Engine Load: 000.00 %
Update Load: 000.00 %
Real FPS: 0
Simulation Running...
Simulation Stopped
FirmWare Uploaded to mega328-109(mega328)
C:/Users/alehx/OneDrive/Desktop/UADY/UAEV 2026/Arquitectura y Organización de Computadoras/controlador_temperatura/controlador_temperatura.hex
Searching for variables... 3 variables found
Searching for Functions... 25 functions found
Mapping Flash to Source... 80 lines mapped

Prueba 3 (el ventiladorno funcionaba porque el diodo fue conectado incorrectamente):

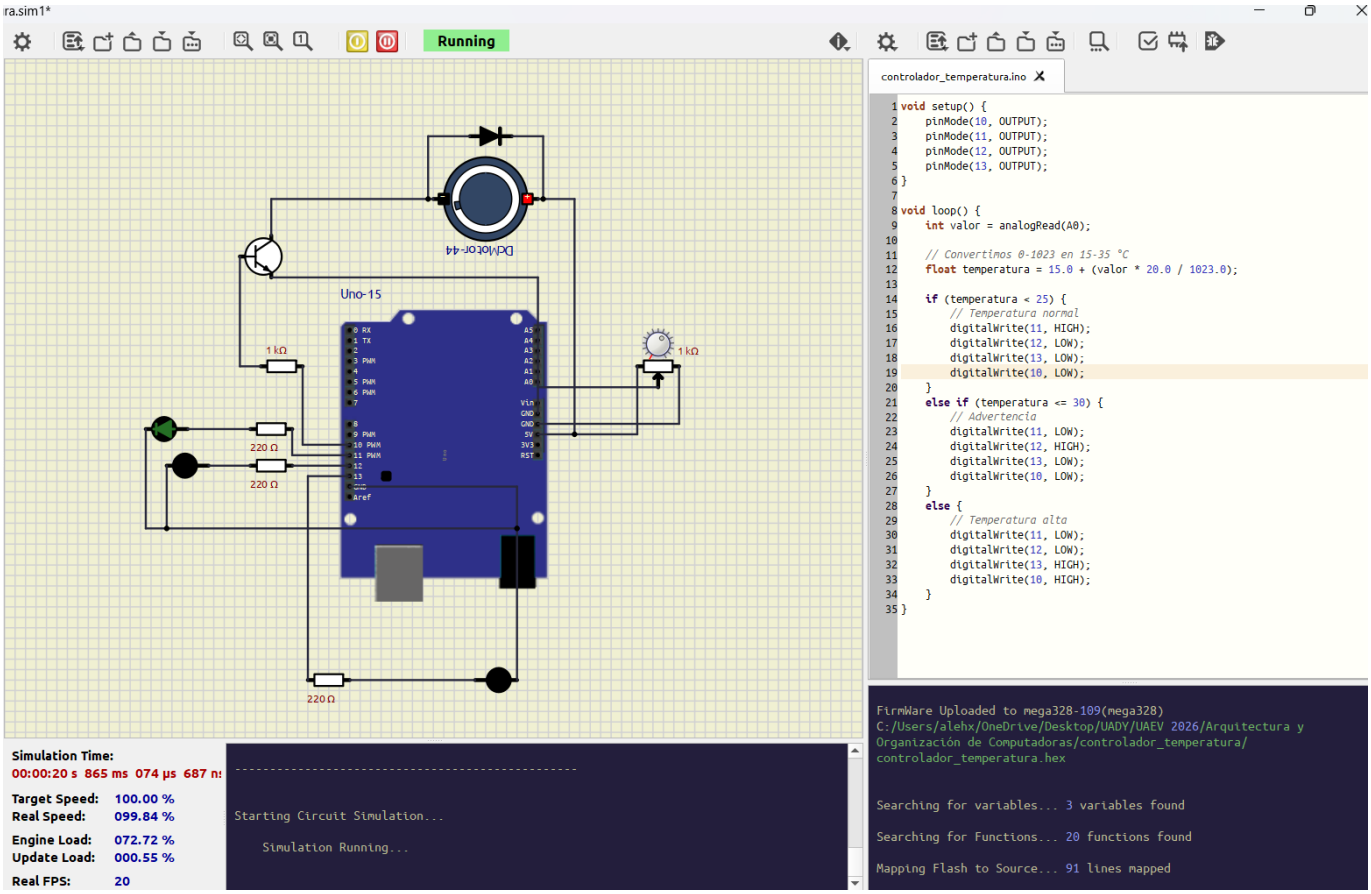
```
void setup() {
    pinMode(10, OUTPUT);
}

void loop() {
    digitalWrite(10, HIGH);
}
```

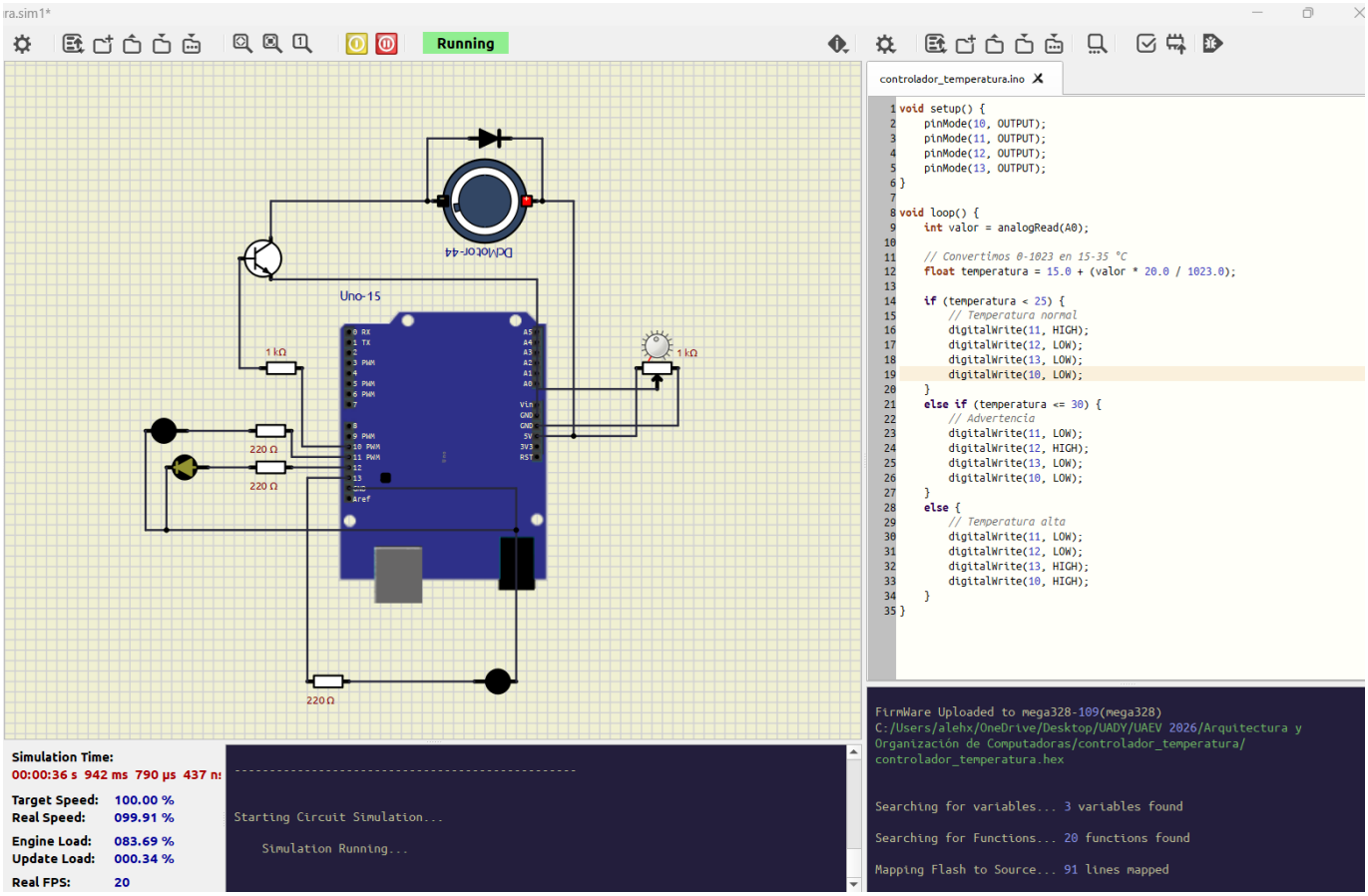


La diferencia entre la imagen previa y la siguiente se debe a un error al conectar el diodo, posteriormente fue corregido para el correcto funcionamiento del dispositivo.

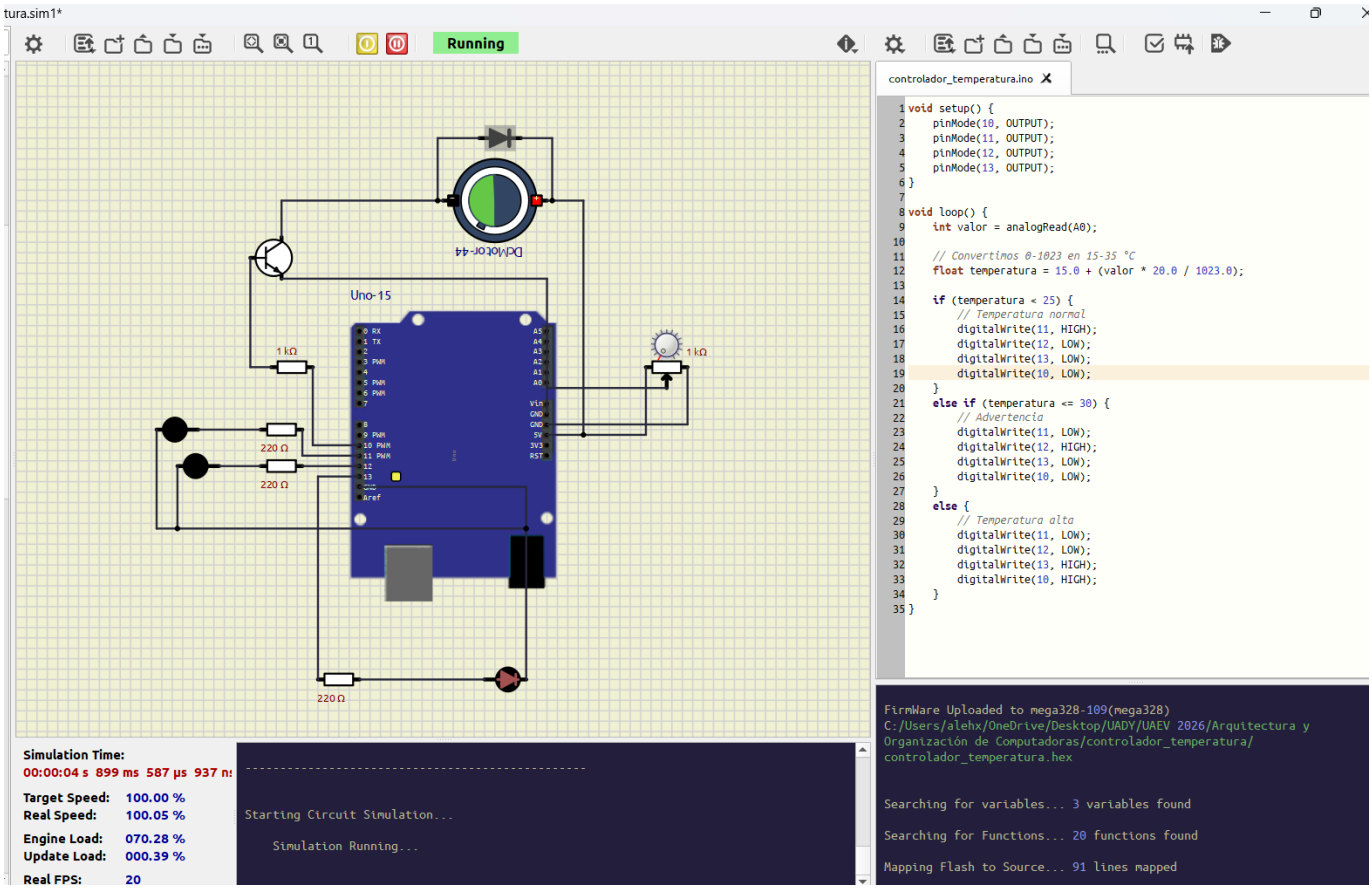
A continuación adjuntamos las imágenes del microcontrolador funcionando correctamente, se observa que el led verde y el led amarillo encienden conforme gira el potenciómetro, simulando un aumento de temperatura en el prototipo, pero el ventilador no llega a encenderse porque no ha rebasado la temperatura establecida:



Enciente en naranja si giramos un poco el potenciómetro:



Aquí podemos observar que el ventilador está funcionando cuando el led se enciende en rojo indicando que se han superado los 30 grados Celsius (en el simulador).



Conclusión:

El desarrollo de este prototipo nos permitió demostrar el funcionamiento básico de un sistema automatizado de monitoreo y control de temperatura utilizando como arquitectura principal un Arduino UNO R3 con microcontrolador ATmega328P, mediante su programación fue posible interpretar una entrada analógica, establecer sus diferentes condiciones de funcionamiento en su código (C++) y gracias a esto, controlar dispositivos de salida de acuerdo con los diferentes valores obtenidos en el entorno de simulación.

Nuestro último tema en la materia fue sobre la interación entre el hardware y el software, y en este proyecto, demostramos cómo la adecuada combinación de hardware y software permite que el microcontrolador reciba información del exterior (por medio de un dispositivo de entrada), procese esta información y tome decisiones para controlar otros dispositivos (como por ejemplo, un ventilador en nuestro prototipo), estableciendo una base para posteriormente desarrollar sistemas automatizados de mayor complejidad como los planteados a lo largo de este proyecto.

Otro aspecto a mencionar es que si bien nuestro prototipo tiene limitaciones y se desarrolla en un entorno de simulación en software, su propósito es simular las variaciones de temperatura y comprender cómo podría evolucionar hacia una implementación física utilizando sensores de temperatura (en lugar de un potenciómetro) y dispositivos más apropiados a las condiciones, como aspersores (en lugar del motor DC), para un mejor desempeño.

REFERENCIAS:

Arduino. (s.f). Language Reference. https://docs.arduino.cc/language-reference/

Microchip Technology Inc. (2015). ATmega328P: 8-bit AVR microcontroller with 32K bytes in-system programmable flash. Microchip Technology Inc. https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

MultimediosTv. (2017). Esta casa sobrevive a incendio gracias a aspersores. https://www.multimediostv.com/telediario/internacional/casa-sobrevive-incendio-gracias-aspersores.html

Materiales proporcionados por el profesor:

González Segura, S. A. (2025). Ciclo de instrucción [Material de lectura de la asignatura Arquitectura y Organización de Computadoras]. Facultad de Matemáticas, Universidad Autónoma de Yucatán.